

15 · Understand & Measure the Prefill Phase

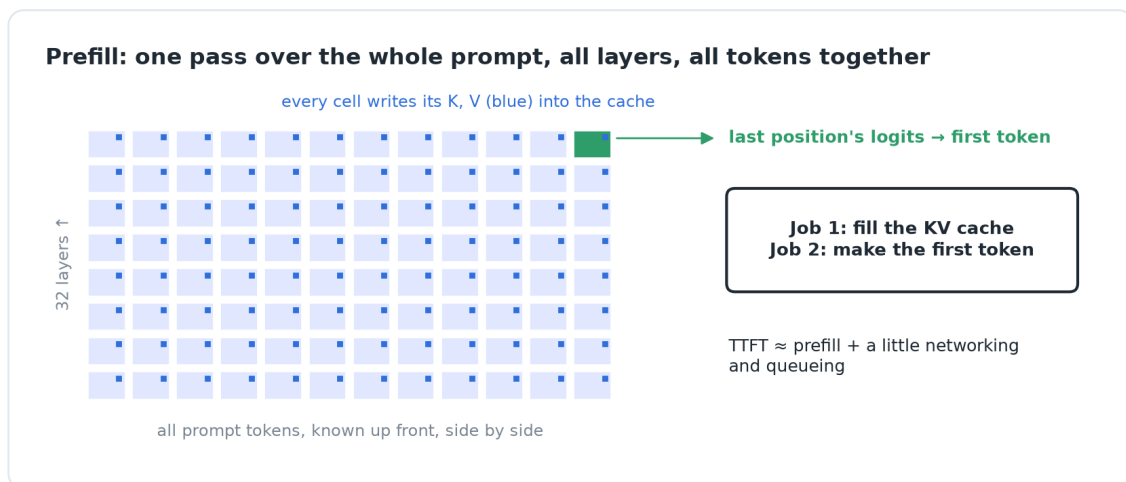
Source: [fanout.sh / inference-eng / 03-02](https://fanout.sh/inference-eng/03-02) · ~6 min

IN SHORT

Most of the pause before the first word is **prefill**: one pass of the model over **every prompt token at once**, which **fills the KV cache** in every layer and **produces the first token**. So $TTFT \approx \text{prefill} + \text{a little networking and queueing}$. Prefill is efficient because each weight byte read is **reused by every prompt token**: ~1 op/byte for one token, ~400 for 500, ~1,000 for 2,000, past the H100's ~300 ridge, so it's **compute-bound**. Napkin: $\text{time} \approx 2 \times \text{params} \times \text{tokens} \div \text{achievable FLOP/s}$: 2,000 tokens on Llama 3 8B ≈ 32 ms at peak, ~65 ms real (~30k tokens/s). Attention adds a term $\propto n^2$ that only matters for long prompts. Measure it with **one request, one output token, a length sweep and a line fit**.

1. What prefill does

With a KV cache, generation has two phases, and prefill is the first. The whole prompt is **known up front**, so no token waits for another: all 2,000 tokens flow through all 32 layers **together**.



Each layer writes keys and values for every prompt token into the cache. At the end, only the last position's logits are used.

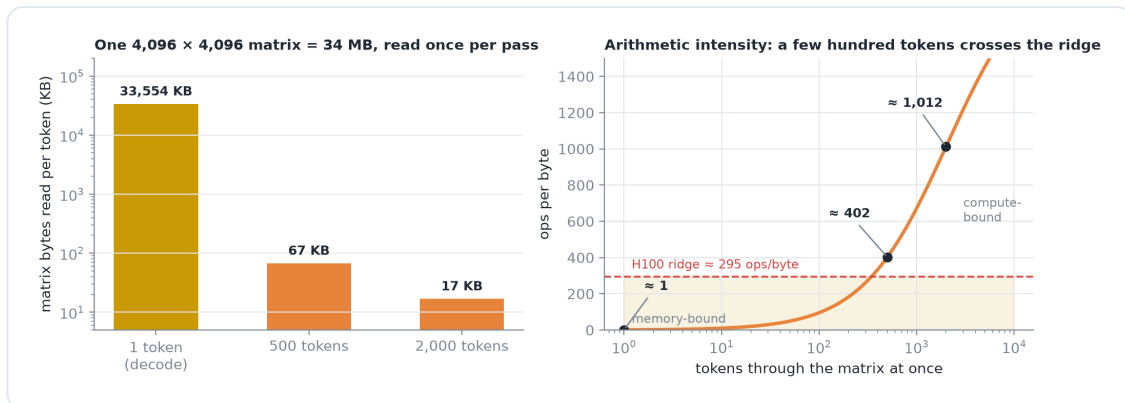
KEY IDEA

Prefill does **two jobs**: fill the cache, and make the first token. That's why **TTFT is mostly prefill**, plus a little networking and queueing.

2. Why prefill is so efficient

Take one $4,096 \times 4,096$ weight matrix: about **34 MB** that the GPU must read from memory to use it.

- **Decode:** one token passes through it. 34 MB read for one small vector of work.
- **Prefill:** 2,000 tokens pass through, and it's read **once**. Every byte is reused 2,000 times.



Arithmetic intensity = math operations ÷ bytes read. Below ~300 ops/byte an H100 waits on memory; above it, the math units are the limit.

INTUITION

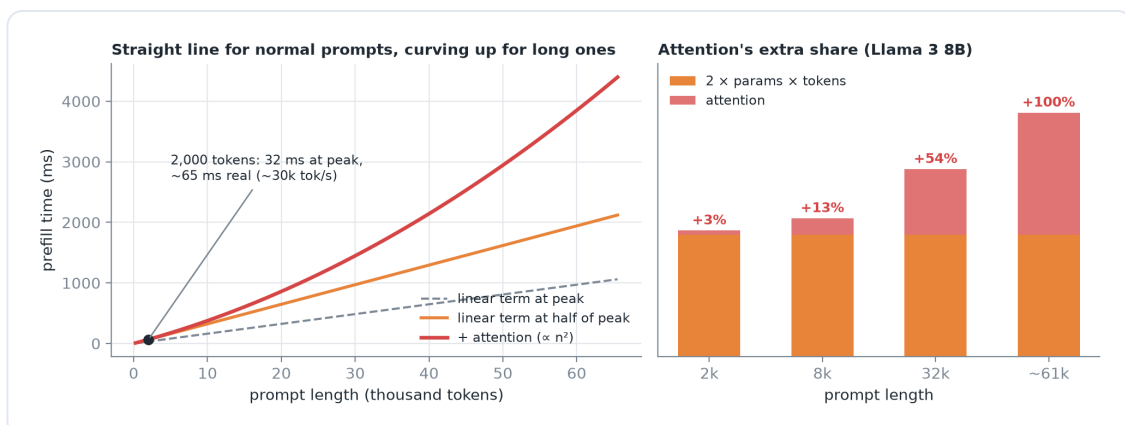
A prompt of **a few hundred tokens** is already enough to make prefill compute-bound.

3. The napkin estimate

A forward pass costs about **2 operations per parameter per token**.

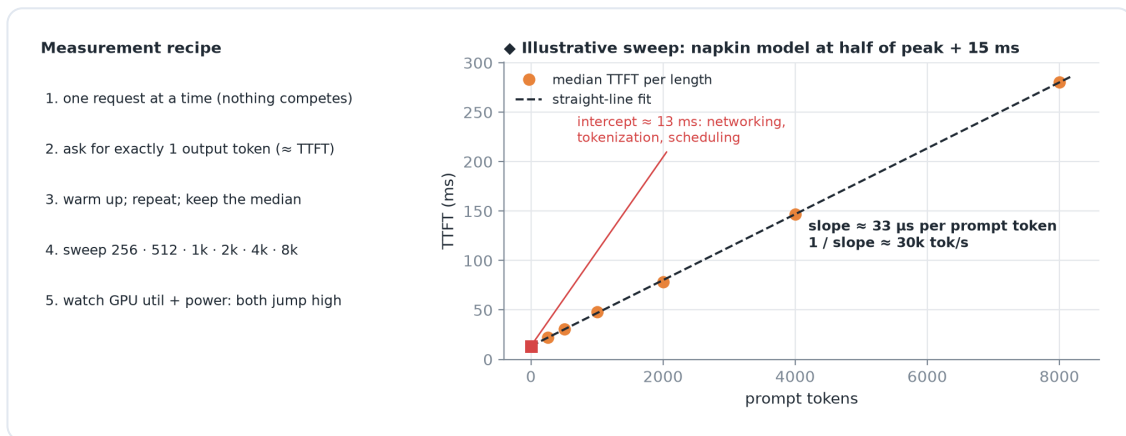
- Llama 3 8B, 2,000-token prompt: $2 \times 8B \times 2,000 \approx$ **32 trillion operations**.
- H100 peak $\approx 1,000$ trillion/s (16-bit) $\rightarrow$ **~32 ms** at peak.
- Real kernels reach **maybe half of peak** $\rightarrow$ expect **~65 ms**, around **30,000 prompt tokens/s**.

One more term: **attention** compares every token with every earlier one, so its work grows with **the square of the prompt length**.



$\approx +3\%$ at 2k tokens, more than half again at 32k, and equal to everything else around 60k. Prefill time is a straight line for normal prompts, curving upward for very long ones.

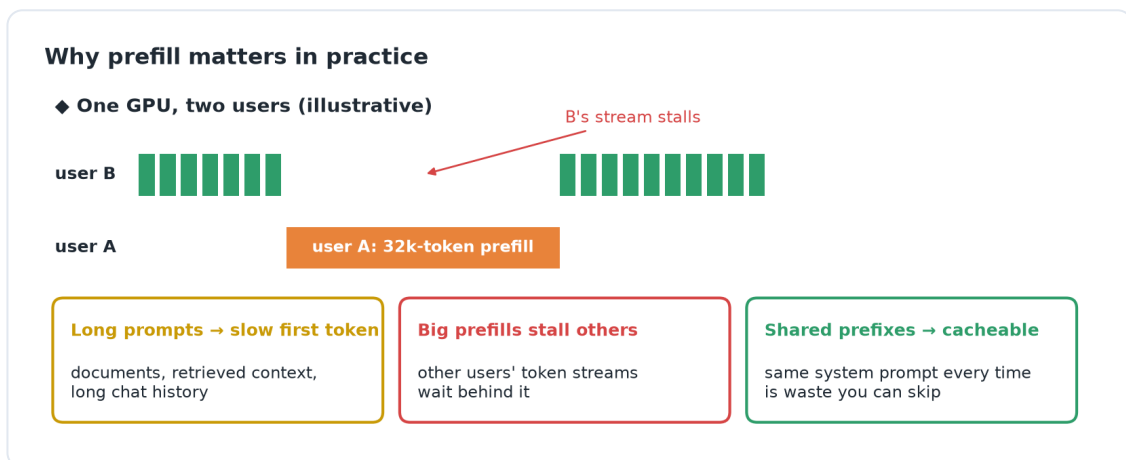
4. Measure it for real



The intercept is the fixed cost (networking, tokenization, scheduling); the slope is time per prompt token, and $1 / \text{slope}$ is your prefill throughput.

Compare the result with the napkin estimate: **far below half of peak means something's wrong**. Watch the GPU too: during a long prefill, **utilization and power draw jump high**, the signature of compute-bound work.

5. Why prefill matters in practice



The fixes are known as

chunked prefill and prefix caching.

6. Recap

Lesson 15 on one card

WHAT IT DOES

whole prompt in one pass
fills the KV cache in every layer
makes the first token ($\approx$ TTFT)

WHY IT'S EFFICIENT

each weight byte reused per token
 ~ 1 op/byte (1 token) $\rightarrow \sim 1,000$ (2k)
past the ~ 300 ridge: compute-bound

NAPKIN

$t \approx 2 \times \text{params} \times \text{tokens} \div \text{FLOP/s}$
2k tokens: 32 ms peak, ~ 65 ms real
+ attention $\propto n^2$ for long prompts

MEASURE

1 request, 1 output token, median
sweep lengths, fit a line
intercept = fixed, $1/\text{slope} = \text{tok/s}$

Interview cheat sheet

Revise from these pages alone. ♦ = my addition beyond the video; everything else is from the lesson. The ~ 300 ops/byte ridge and the at-peak prefill estimate are in 01, derivation 2; MFU in 09.

Say it in 30 seconds

Prefill runs the whole prompt through the model in one pass, writing every token's K and V into the cache in every layer, and the last position's logits give the first token, so TTFT is mostly prefill. Each weight byte is reused by every prompt token, so a few hundred tokens push arithmetic intensity past the ~ 300 ridge and prefill is compute-bound. Estimate it as $2 \times \text{params} \times \text{tokens} \div \text{achievable FLOP/s}$: about 65 ms for 2k tokens on Llama 3 8B at half of H100 peak, plus an n^2 attention term for long prompts. Measure with single requests, $\text{max_tokens} = 1$, warm-up and medians over a length sweep, then fit a line.

Only numbers worth memorizing

- **Llama 3 8B, 2k-token prompt on an H100: ~ 65 ms** (half of peak), **$\sim 30\text{k}$ tokens/s**; attention adds $\sim 3\%$ at 2k, $\sim +54\%$ at 32k.

Derive, don't memorize

1. Intensity of one matmul with T tokens

FLOPs: $2Th^2$. Bytes: the h^2 weights once, plus T input and T output activations (2 bytes each).

$$AI = \frac{2Th^2}{2h^2 + 4Th} = \frac{Th}{h + 2T} \quad T = 1 : 1, \quad 500 : \approx 400, \quad 2000 : \approx 1000$$

INTUITION

For small T the weights dominate the bytes, so $AI \approx T$: every extra token is nearly free math on bytes already read. ♦ For huge T it saturates near $h/2$, because activations dominate.

2. Prefill time from the FLOPs

$$t \approx \frac{2 \cdot N \cdot n}{\text{MFU} \times \text{peak}} = \frac{2 \times 8\text{B} \times 2000}{0.5 \times 989 \text{ TFLOPS}} \approx 65 \text{ ms} \Rightarrow \frac{2000}{65 \text{ ms}} \approx 30\text{k tok/s}$$

3. When attention starts to matter

Causal attention is about half of an $n \times n$ square per layer (scores plus value mixing), against $2N$ per token for the weights.

$$\frac{2Ln^2h}{2Nn} = \frac{Lhn}{N} = \frac{32 \times 4096 \times n}{8\text{B}} \approx 3\% (2\text{k}), \quad 54\% (32\text{k}), \quad 100\% (\approx 61\text{k})$$

4. Read the sweep as a line

$$\text{TTFT}(n) \approx t_0 + \frac{n}{\text{prefill tok/s}} \quad \text{intercept} = t_0, \quad \frac{1}{\text{slope}} = \text{throughput}$$

What the sweep tells you

Feature	Meaning
Intercept	Fixed cost: networking, tokenization, scheduling
Slope	Time per prompt token
1 / slope	Prefill throughput (tokens/s)
◆ Upward curve at long lengths	The n^2 attention term
High GPU util and power	Compute-bound work, as expected

Symptom → diagnosis

Symptom	Diagnosis
Measured throughput far below half the napkin	Something's wrong: investigate the setup
TTFT numbers jump around	Concurrent requests, no warm-up, or no median
Other users' streams stall during long prompts	A big prefill occupies the GPU (fix: chunked prefill)
The same long system prompt is slow every time	Shared prefix recomputed (fix: prefix caching)

Rapid-fire Q&A

Question	Crisp answer
What two jobs does prefill do?	Fills the KV cache and produces the first token.
Why can prefill run all tokens at once?	The whole prompt is known up front; no token waits for another.
Why is prefill compute-bound?	Each weight byte is reused by every prompt token.
Why set max_tokens = 1 when measuring?	The response time is then basically TTFT.
When does attention dominate prefill?	Around 60k tokens on Llama 3 8B; ~3% at 2k.

TRAP

- Measuring TTFT under concurrent load: other requests contaminate it.
- Estimating at peak FLOP/s: real kernels reach about half.
- Assuming prefill is linear forever: attention bends it upward at long prompts.